

Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



A Lab Report
on
“Operating Systems Lab-II”

[Code No.: COMP 342]

Submitted by

Prajwal Ghimire

Roll No.: 22

Submitted to

Mr. Rabina Shrestha

Department of Computer Science and Engineering

January 11, 2026

Understanding fork() and Process Creation

Objective

To study how `fork()` creates new processes and understand why multiple `fork()` calls increase the number of processes exponentially.

1 Program 1: Process Creation

1.1 Part A – Two `fork()` calls

1.1.1 Modification

The program was modified to include exactly two `fork()` calls.

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     fork();
6     fork();
7
8     printf("Hello World!!!\n");
9     return 0;
10 }
```

Figure 1: Source Code with two `fork()`.

1.1.2 Output Analysis

When running this program "Hello World!!!" is printed 4 times.

```
@ > ~/os/lab/lab_2 > on 🐱 master !1 ?2
> ./fork
Hello World!!!
Hello World!!!
Hello World!!!
Hello World!!!
```

Figure 2: Output of two `fork()`

1.1.3 Process Tree Diagram and Analysis

Process Tree Diagram:

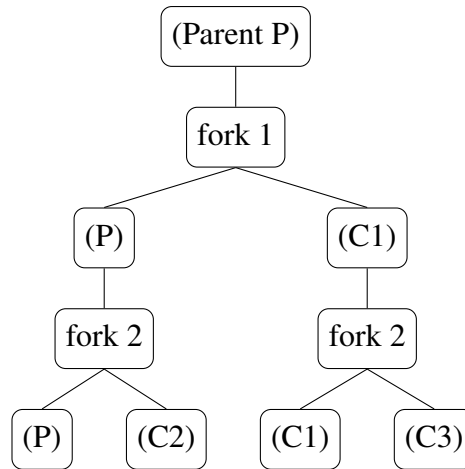


Figure 3: Process hierarchy after successive fork calls

Analysis:

- **First fork():** The original parent process (P) creates a new child (C1). (Total: 2 processes).
- **Second fork():** Both the parent (P) and the first child (C1) execute this instruction. P creates C2, and C1 creates C3. (Total: 4 processes).

Therefore, the print statement is executed by all 4 processes.

1.2 Part B – Three `fork()` calls

1.2.1 Modification

The program was modified to include exactly three `fork()` calls.

```
1 #include <stdio.h>
1 #include <unistd.h>
2
3 int main() {
4
5     fork();
6     fork();
7     fork();
8
9     printf("Hello World!!!\n");
10    return 0;
11 }
```

Figure 4: Source Code with three `fork()`.

1.2.2 Output Analysis

When running this program "Hello World!!!" is printed 8 times.

```
@ > ~ /os/lab/lab_2 > on master !1 ?2
> ./3fork
Hello World!!!
Hello World!!!
Hello World!!!
Hello World!!!
Hello World!!!
Hello World!!!
Hello World!!!
Hello World!!!
```

Figure 5: Output of three `fork()`

1.2.3 Process Tree Diagram and Analysis

Process Tree Diagram:

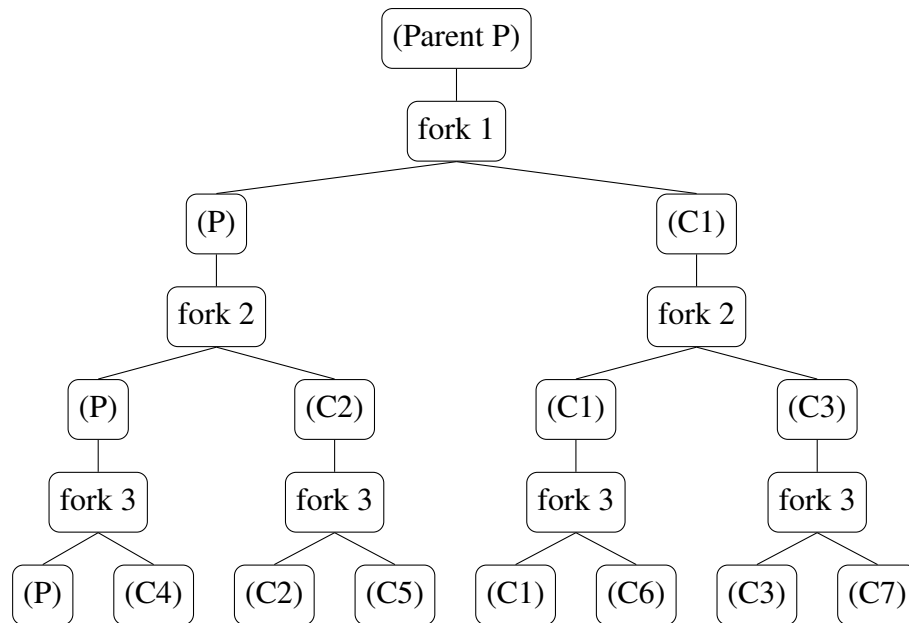


Figure 6: Process hierarchy after three successive fork calls

Analysis:

- **First fork():** The original parent process (P) creates one child (C1). (Total: 2 processes).
- **Second fork():** Both existing processes execute `fork()`. P creates C2 and C1 creates C3. (Total: 4 processes).
- **Third fork():** All four processes execute `fork()`. Each creates one new child process. (Total: 8 processes).

Therefore, each `fork()` duplicates every currently running process, resulting in the number of processes doubling at each step.

2 Program 2: Understanding `fork()` with PID

Objective: A program that shows the use of forking and identifying Parent vs Child processes with PIDs.

```
20 #include <stdio.h>
19 #include <unistd.h>
18
17 int main() {
16     int pid;
15     printf("I am the parent Process with ID %d\n", getpid());
14
13     printf("Here I am before the use of forking\n");
12
11     pid = fork();
10
9     printf("Here I am just after forking\n");
8
7     if (pid == 0) {
6         printf("I am a child process with ID %d\n", getpid());
5     }
4     else {
3         printf("I am the parent process with PID %d\n", getpid());
2     }
1     return 0;
21 }
```

Figure 7: PID identification code

Output:

```
@ > ~ / os / lab / lab_2 > on master !1 ?2
> ./pid
I am the parent Process with ID 10800
Here I am before the use of forking
Here I am just after forking
Here I am just after forking
I am the parent process with PID 10800
I am a child process with ID 10801
```

Figure 8: Fork output with PID

Questions

Q1: Explain the difference between `pid == 0` and `pid > 0`. Which one corresponds to the child and the parent?

Answer:

- `pid == 0`: This corresponds to the **Child Process**. The `fork()` system call returns 0 to the newly created child process so it knows it is the child.
- `pid > 0`: This corresponds to the **Parent Process**. The `fork()` system call returns the Process ID (a positive integer) of the new child to the parent process.

Q2: Why do the parent and child processes print their own PID differently?

Answer: They print different PIDs because they are two separate processes handled by the operating system. The parent retains its original PID (394681), while the OS assigns a new unique PID (394682) to the child process.

Q3: Explain why the output order is parent lines first, child lines after. Can the order change? Why?

Answer:

- **Current Order:** The parent printed first because the CPU scheduler allowed the parent to continue execution immediately after the fork while the child was being initialized.
- **Can it change?** Yes, the order is non-deterministic.
- **Why?** It depends on the OS Scheduling algorithm. If a context switch occurs right after `fork()`, the child might run before the parent.